visualize_sensitivity_study

February 12, 2026

1 Figure 15. Experimental results in sensitivity analysis.

[1]:
```
import torch
import os
import numpy as np
import matplotlib.pyplot as plt
from collections import defaultdict
from Suika import analyze_job_statistics

# --- Configuration ---
scenarios_to_plot = {
    "bdw": {
        "name": "Inter-Node Bandwidth",
        "cases": ["clusterA", "clusterB", "clusterC"],
        "case_map": {
            "clusterA": "8x400",
            "clusterB": "4x400",
            "clusterC": "10"
        },
        "sub_caption": "(a) Inter-Node Bandwidth (Gbps)"
    },
    "int": {
        "name": "Job Intensity",
        "cases": ["int0.75", "clusterA", "int1.25", "int1.5", "int2.0"],
        "case_map": {
            "int0.75": "x0.75",
            "clusterA": "x1.0",
            "int1.25": "x1.25",
            "int1.5": "x1.5",
            "int2.0": "x2.0",
        },
        "sub_caption": "(b) Job Intensity"
    },
    "lj": {
        "name": "Job Composition",
        "cases": ["lj0.5", "clusterA", "lj1.5",
                ],
```

```
        "case_map": {
            "lj0.5": "4:1",
            "clusterA": "3:2",
            "lj1.5": "2:3",
        },
        "sub_caption": "(c) Job Composition (S:(M+L))"
    }
}


policies = ["suika", "sia", "rubick", "fcfs"]
policy_map = {
    "suika": "Suika",
    "sia": "Sia",
    "rubick": "Rubick",
    "dp": "Suika-DP",
    "metis": "Suika-Metis",
    "fcfs": "Non-Elastic",
}

title_offset = -0.25


all_stats_by_scenario = {}

print("Loading data for all scenarios...")
for scenario_key, config in scenarios_to_plot.items():
    case_stats = defaultdict(lambda: defaultdict(dict))
    print(f"\nLoading data for scenario: {config['name']}")
    for case in config["cases"]:
        for policy in policies:
            file_path = f"../results/{case}/{policy}-{case}.pth"
            try:
                stats = torch.load(file_path, weights_only=False)
                analyzed = analyze_job_statistics(stats)
                avg_jct = analyzed.get("avg_jct")
                avg_w_jct = analyzed.get("avg_w_jct")

                if avg_jct is not None and avg_w_jct is not None:
                    case_stats[case][policy] = {
                        'jct': avg_jct,
                        'w_jct': avg_w_jct
                    }
                    print(f"  - Loaded {case}/{policy}: Avg JCT = {avg_jct:.
 ↪2f}, Avg wJCT = {avg_w_jct:.2f}")
                else:
```

```
                    print(f"  - WARNING: Could not find avg_jct or avg_w_jct␣
↪for {case}/{policy}")

            except FileNotFoundError:
                print(f"  - SKIPPING: File not found at {file_path}")
            except Exception as e:
                print(f"  - ERROR: Failed to load or process {file_path}: {e}")
    all_stats_by_scenario[scenario_key] = case_stats

print("\nData loading complete for all scenarios.")
```

```
/home/fake/.local/lib/python3.10/site-packages/tqdm/auto.py:21: TqdmWarning:
IProgress not found. Please update jupyter and ipywidgets. See
https://ipywidgets.readthedocs.io/en/stable/user_install.html
  from .autonotebook import tqdm as notebook_tqdm
Loading data for all scenarios…

Loading data for scenario: Inter-Node Bandwidth
  - Loaded clusterA/suika: Avg JCT = 2127.41, Avg wJCT = 3531.14
  - Loaded clusterA/sia: Avg JCT = 2735.18, Avg wJCT = 4825.25
  - Loaded clusterA/rubick: Avg JCT = 2786.77, Avg wJCT = 4746.97
  - Loaded clusterA/fcfs: Avg JCT = 3561.93, Avg wJCT = 5185.36
  - Loaded clusterB/suika: Avg JCT = 2508.72, Avg wJCT = 4226.24
  - Loaded clusterB/sia: Avg JCT = 3162.22, Avg wJCT = 5738.25
  - Loaded clusterB/rubick: Avg JCT = 3466.41, Avg wJCT = 5560.57
  - Loaded clusterB/fcfs: Avg JCT = 3877.33, Avg wJCT = 5757.64
  - Loaded clusterC/suika: Avg JCT = 2854.63, Avg wJCT = 4975.23
  - Loaded clusterC/sia: Avg JCT = 3014.93, Avg wJCT = 5318.81
  - Loaded clusterC/rubick: Avg JCT = 3042.58, Avg wJCT = 5147.64
  - Loaded clusterC/fcfs: Avg JCT = 3737.30, Avg wJCT = 5613.64

Loading data for scenario: Job Intensity
  - Loaded int0.75/suika: Avg JCT = 1866.65, Avg wJCT = 3084.17
  - Loaded int0.75/sia: Avg JCT = 2619.33, Avg wJCT = 4632.31
  - Loaded int0.75/rubick: Avg JCT = 2498.48, Avg wJCT = 4203.26
  - Loaded int0.75/fcfs: Avg JCT = 3548.51, Avg wJCT = 5160.83
  - Loaded clusterA/suika: Avg JCT = 2127.41, Avg wJCT = 3531.14
  - Loaded clusterA/sia: Avg JCT = 2735.18, Avg wJCT = 4825.25
  - Loaded clusterA/rubick: Avg JCT = 2786.77, Avg wJCT = 4746.97
  - Loaded clusterA/fcfs: Avg JCT = 3561.93, Avg wJCT = 5185.36
  - Loaded int1.25/suika: Avg JCT = 2793.67, Avg wJCT = 4529.19
  - Loaded int1.25/sia: Avg JCT = 3090.74, Avg wJCT = 5559.94
  - Loaded int1.25/rubick: Avg JCT = 3359.26, Avg wJCT = 5315.69
  - Loaded int1.25/fcfs: Avg JCT = 3910.48, Avg wJCT = 5594.88
  - Loaded int1.5/suika: Avg JCT = 3143.44, Avg wJCT = 5352.92
  - Loaded int1.5/sia: Avg JCT = 3496.69, Avg wJCT = 6218.49
  - Loaded int1.5/rubick: Avg JCT = 3452.18, Avg wJCT = 5592.95
```

```
  - Loaded int1.5/fcfs: Avg JCT = 4644.67, Avg wJCT = 6631.22
  - Loaded int2.0/suika: Avg JCT = 3576.25, Avg wJCT = 6335.95
  - Loaded int2.0/sia: Avg JCT = 3766.54, Avg wJCT = 6661.75
  - Loaded int2.0/rubick: Avg JCT = 4221.41, Avg wJCT = 7140.48
  - Loaded int2.0/fcfs: Avg JCT = 5264.31, Avg wJCT = 7922.81

Loading data for scenario: Job Composition
  - Loaded lj0.5/suika: Avg JCT = 2906.02, Avg wJCT = 4287.54
  - Loaded lj0.5/sia: Avg JCT = 3446.99, Avg wJCT = 5580.51
  - Loaded lj0.5/rubick: Avg JCT = 3609.56, Avg wJCT = 5389.25
  - Loaded lj0.5/fcfs: Avg JCT = 5337.14, Avg wJCT = 6715.99
  - Loaded clusterA/suika: Avg JCT = 2127.41, Avg wJCT = 3531.14
  - Loaded clusterA/sia: Avg JCT = 2735.18, Avg wJCT = 4825.25
  - Loaded clusterA/rubick: Avg JCT = 2786.77, Avg wJCT = 4746.97
  - Loaded clusterA/fcfs: Avg JCT = 3561.93, Avg wJCT = 5185.36
  - Loaded lj1.5/suika: Avg JCT = 2121.04, Avg wJCT = 3095.28
  - Loaded lj1.5/sia: Avg JCT = 2651.56, Avg wJCT = 4057.25
  - Loaded lj1.5/rubick: Avg JCT = 2600.62, Avg wJCT = 3866.04
  - Loaded lj1.5/fcfs: Avg JCT = 3084.57, Avg wJCT = 4155.33

Data loading complete for all scenarios.
```

[2]:
```python
def plot_metric_on_ax(ax, stats, metric_key, y_label, scenario_name, case_map,␣
↪sub_caption, policy_map, policies, show_y_label=True,␣
↪sub_caption_y=title_offset):
    """Generates a grouped bar chart for a given metric on a specific Axes␣
↪object."""

    valid_cases = {}
    for case, policies_data in stats.items():
        cluster_metrics = {}
        for p, metrics in policies_data.items():
            if metric_key in metrics:
                cluster_metrics[p] = metrics[metric_key]
        if cluster_metrics:
            valid_cases[case] = cluster_metrics

    if not valid_cases:
        ax.text(0.5, 0.5, f"No data for {metric_key}", ha='center', va='center')
        return

    case_names = list(valid_cases.keys())
    policy_names = sorted(list(set(p for c in valid_cases.values() for p in c.
↪keys())), key=policies.index)

    x = np.arange(len(case_names))
    n_policies = len(policy_names)
```

```
    width = 0.8 / n_policies

    policy_styles = {
        "suika":  {"color": "#34495e", "hatch": ""},
        "sia":    {"color": "#e67e22", "hatch": "/"},
        "rubick": {"color": "#f1c40f", "hatch": "x"},
        "fcfs":   {"color": "#e74c3c", "hatch": "-"},
    }

    for i, policy in enumerate(policy_names):
        metric_values = [valid_cases[c].get(policy, np.nan) for c in case_names]
        style = policy_styles.get(policy, {"color": "gray", "hatch": ""})

        offset = (i - (n_policies - 1) / 2) * width
        ax.bar(x + offset, metric_values, width, label=policy_map.get(policy,␣
↪policy.upper()),
               color=style["color"], hatch=style["hatch"], edgecolor='white')

    for i, case in enumerate(case_names):
        cluster_metrics = valid_cases[case]
        if not cluster_metrics:
            continue

        min_metric = min(val for val in cluster_metrics.values() if val is not␣
↪np.nan)

        # for j, policy in enumerate(policy_names):
        #     if policy in cluster_metrics and cluster_metrics[policy] is not␣
↪np.nan:
        #         metric_value = cluster_metrics[policy]
        #         relative_perf = metric_value / min_metric
        #         bar = ax.containers[j][i]
        #         ax.annotate(f'{relative_perf:.2f}x',
        #                     xy=(bar.get_x() + bar.get_width() / 2, bar.
↪get_height()),
        #                     xytext=(0, 3),
        #                     textcoords="offset points",
        #                     ha='center', va='bottom',
        #                     fontsize=10, fontweight='bold')

    if show_y_label:
        ax.set_ylabel(y_label, fontsize=16, fontweight='bold')
    ax.set_xticks(x)
    ax.set_xticklabels([case_map.get(c, c.upper()) for c in case_names],␣
↪fontsize=14)
    ax.tick_params(axis='y', labelsize=12)
```

```
    # if scenario_name == "Large Job Portion / XL Job":
    #     ax.axvline(x=2.5, color='gray', linestyle='-.', linewidth=2)

    current_ylim = ax.get_ylim()
    if scenario_name in ["Inter-Node Bandwidth", "Job Intensity", "Job␣
 ↪Composition"]:
        ax.set_ylim(0, 5500)
    else:
        ax.set_ylim(current_ylim[0], current_ylim[1] * 1.1)

    ax.grid(True, axis='y', linestyle='--', alpha=0.6)
    ax.set_title(sub_caption, fontsize=16, fontweight='bold', y=sub_caption_y)

# --- Generate Combined Plot ---
import matplotlib.gridspec as gridspec

# Create a figure with a specific size and layout
fig = plt.figure(figsize=(18, 3))
gs = gridspec.GridSpec(1, 3, width_ratios=[3, 5.5, 3], wspace=0.15)

# Create subplots from the grid spec
ax1 = plt.subplot(gs[0])
ax2 = plt.subplot(gs[1])
ax3 = plt.subplot(gs[2])
axes = [ax1, ax2, ax3]

metric_to_plot = 'jct'
y_label = 'Average JCT (s)'

# Plotting for each scenario
plot_metric_on_ax(axes[0], all_stats_by_scenario['bdw'], metric_to_plot,␣
 ↪y_label, scenarios_to_plot['bdw']['name'],␣
 ↪scenarios_to_plot['bdw']['case_map'],␣
 ↪scenarios_to_plot['bdw']['sub_caption'], policy_map, policies,␣
 ↪show_y_label=True)
plot_metric_on_ax(axes[1], all_stats_by_scenario['int'], metric_to_plot,␣
 ↪y_label, scenarios_to_plot['int']['name'],␣
 ↪scenarios_to_plot['int']['case_map'],␣
 ↪scenarios_to_plot['int']['sub_caption'], policy_map, policies,␣
 ↪show_y_label=False)
plot_metric_on_ax(axes[2], all_stats_by_scenario['lj'], metric_to_plot,␣
 ↪y_label, scenarios_to_plot['lj']['name'],␣
 ↪scenarios_to_plot['lj']['case_map'], scenarios_to_plot['lj']['sub_caption'],␣
 ↪policy_map, policies, show_y_label=False)

# Create a single legend for the entire figure
handles, labels = axes[0].get_legend_handles_labels()
```

```
fig.legend(handles, labels, loc='upper center', ncol=len(policies),␣
 ↪fontsize=14, bbox_to_anchor=(0.5, 1.05))

# Adjust layout and save the figure
fig.tight_layout(rect=[0, 0.03, 1, 0.95])

# Save the figure
fig_dir = "../fig"
os.makedirs(fig_dir, exist_ok=True)
filename = f"{fig_dir}/sensitivity_study_row1.pdf"
plt.savefig(filename, dpi=300, bbox_inches='tight')
print(f"Combined plot saved as '{filename}'")

plt.show()
```

```
/tmp/ipykernel_200624/497360081.py:102: UserWarning: This figure includes Axes
that are not compatible with tight_layout, so results might be incorrect.
  fig.tight_layout(rect=[0, 0.03, 1, 0.95])
```

```
Combined plot saved as '../fig/sensitivity_study_row1.pdf'
```



2 Visualize Second Row of Subgraphs

(1). Sensitivity Study (d) scheduling hyperparameters.

Data Format: f“results/th-exp-ablation/suika-th-exp-{lbd:.2f}.pth”

lbd in [0.25, 0.5, 0.75, 1.0, 1.25, 1.5, 1.75, 2.0]

Metircs: JCT (algin to left axis), Utils (align to right axis). plot chart in the same graph, use two different line stype and markers.

X-axis should be lbd (_{th}), err not sure if latex can be supported. restrict x lim tightly by 0.1 value margin, and specify ticks explicitly.

(2). Sensitivity Study (e) Base Policy

Data Format: f“results/base-policy-ablation/{policy}.pth”

policy in [fcfs, sjf]

Plot a thin, clustered bar chart right to the figure(d), comparing JCT and Queueing delay ("avg_queuing_delay" field).

Use a legend and hatch for fcfs, sjf; use JCT/Queueing Delay as x-label

Note: fcfs should be named FIFO, and sjf SJF.

(3). Sensitivity Study (f) Trace

Data Format: f"results/{trace}/{policy}-{trace}.pth"

trace in ["clusterA", "philly2", "helios_venus", "helios_earth"]

policy in ["suika", "sia", "rubick", "fcfs"]

refer to policy name mapping, color, hatches as in previous subfigures.

plot JCT as cluster bar chart (all in one)

[3]:
```
# --- Configuration for Sensitivity Study ---
lbd_values = [0.25, 0.5, 0.75, 1.0, 1.25, 1.5, 1.75, 2.0]
th_exp_ablation_dir = "th-exp-ablation"
sensitivity_stats = defaultdict(list)
from matplotlib import markers
from Suika import parse_cluster_config
config_path = "../configs/cluster64.json"
total_n_acc = parse_cluster_config(config_path)[0].get_total_acc_num()

# --- Configuration for Base Policy Study (e) ---
base_policy_ablation_dir = "sjf-ablation"
base_policies = ["fcfs", "sjf", "suika-fcfs", "suika-sjf"]
base_policy_stats = defaultdict(dict)
base_policy_map = {
    "fcfs": "FIFO",
    "suika-fcfs": "Suika-FIFO",
    "sjf": "SJF",
    "suika-sjf": "Suika-SJF"
}

# --- Configuration for Trace Study (f) ---
trace_study_traces = ["clusterA", "philly2", "helios_venus", "helios_uranus"]
trace_study_policies = ["suika", "sia", "rubick", "fcfs"]
trace_study_stats = defaultdict(lambda: defaultdict(dict))
trace_map = {
    "clusterA": "Philly1",
    "philly2": "Philly2",
    "helios_venus": "Helios1",
    "helios_uranus": "Helios2",
}

# --- JCT + UTIL Analysis Function ---
```

```
def analyze_and_extract_data_event_driven(stats, total_n_acc):
    analyzed_stats_base = analyze_job_statistics(stats)
    n_job = len(stats)
    gpu_time = analyzed_stats_base["avg_gpu_time"] * n_job
    make_span = analyzed_stats_base["make_span"]
    cluster_utilization = (gpu_time / (make_span * total_n_acc)) * 100 if␣
 ↪make_span > 0 else 0

    return analyzed_stats_base['avg_jct'], cluster_utilization

# --- Data Loading ---
print("Loading data for sensitivity study...")
for lbd in lbd_values:
    file_path = f"../results/{th_exp_ablation_dir}/suika-th-exp-{lbd:.2f}.pth"
    try:
        stats = torch.load(file_path, weights_only=False)
        avg_jct, utilization = analyze_and_extract_data_event_driven(stats,␣
 ↪total_n_acc)
        sensitivity_stats['lbd'].append(lbd)
        sensitivity_stats['jct'].append(avg_jct)
        sensitivity_stats['util'].append(utilization)
        print(f"  - Loaded lbd={lbd:.2f}: JCT={avg_jct:.2f}, Util={utilization:.
 ↪2f}%")
    except FileNotFoundError:
        print(f"  - SKIPPING: File not found at {file_path}")
    except Exception as e:
        print(f"  - ERROR: Failed to load or process {file_path}: {e}")

# --- Data Loading for Base Policy Study (e) ---
print("\nLoading data for base policy sensitivity study...")
for policy in base_policies:
    file_path = f"../results/{base_policy_ablation_dir}/{policy}.pth"
    try:
        stats = torch.load(file_path, weights_only=False)
        analyzed = analyze_job_statistics(stats)
        avg_jct = analyzed.get("avg_jct")
        avg_queuing_delay = analyzed.get("avg_queuing_delay")
        if avg_jct is not None:
            base_policy_stats[policy]['jct'] = avg_jct
            print(f"  - Loaded {policy}: JCT={avg_jct:.2f}")
        else:
            print(f"  - WARNING: Could not find avg_jct or avg_queuing_delay␣
 ↪for {policy}")
    except FileNotFoundError:
        print(f"  - SKIPPING: File not found at {file_path}")
    except Exception as e:
        print(f"  - ERROR: Failed to load or process {file_path}: {e}")
```

```
# --- Data Loading for Trace Study (f) ---
print("\nLoading data for trace study...")
for trace in trace_study_traces:
    for policy in trace_study_policies:
        file_path = f"../results/{trace}/{policy}-{trace}.pth"
        try:
            stats = torch.load(file_path, weights_only=False)
            analyzed = analyze_job_statistics(stats)
            avg_jct = analyzed.get("avg_jct")
            if avg_jct is not None:
                trace_study_stats[trace][policy] = {'jct': avg_jct}
                print(f"  - Loaded {trace}/{policy}: JCT={avg_jct:.2f}")
            else:
                print(f"  - WARNING: Could not find avg_jct for {trace}/
 ↪{policy}")
        except FileNotFoundError:
            print(f"  - SKIPPING: File not found at {file_path}")
        except Exception as e:
            print(f"  - ERROR: Failed to load or process {file_path}: {e}")

# --- Plotting ---
import matplotlib.gridspec as gridspec

# Create a figure with a specific size and layout for (d), (e), and (f)
fig = plt.figure(figsize=(18, 3))
gs = gridspec.GridSpec(1, 3, width_ratios=[3.5, 2, 4], wspace=0.3)

# --- Subplot (d) ---
ax1 = plt.subplot(gs[0])

# Plot JCT
color = 'tab:blue'
ax1.set_xlabel(r'$\lambda_{\tau}$', fontsize=16, fontweight='bold')
ax1.xaxis.set_label_coords(-0.03, -0.02, transform=ax1.transAxes) # Adjust␣
 ↪position
ax1.set_ylabel('Average JCT (s)', color=color, fontsize=14, fontweight='bold')
ax1.plot(sensitivity_stats['lbd'], sensitivity_stats['jct'], color=color,␣
 ↪marker='o', linestyle='-', label='JCT', markersize=8)
ax1.tick_params(axis='y', labelcolor=color, labelsize=12)
ax1.grid(True, axis='y', linestyle='--', alpha=0.6)

# Create a second y-axis for Utilization
ax2 = ax1.twinx()
color = 'tab:red'
ax2.set_ylabel('Utilization (%)', color=color, fontsize=14, fontweight='bold')
```

```
ax2.plot(sensitivity_stats['lbd'], sensitivity_stats['util'], color=color,␣
 ↪marker='D', linestyle='--', label='Utilization', markersize=8)
ax2.tick_params(axis='y', labelcolor=color, labelsize=12)

# Set x-axis ticks and limits
ax1.set_xticks(lbd_values)
ax1.set_xlim(min(lbd_values) - 0.1, max(lbd_values) + 0.1)
ax1.tick_params(axis='x', labelsize=12)

# Add legend for (d)
lines, labels = ax1.get_legend_handles_labels()
lines2, labels2 = ax2.get_legend_handles_labels()
ax2.legend(lines + lines2, labels + labels2, loc='center right')
ax1.set_title(r'(d) Expansion-Aggressiveness Knob $\lambda_{\tau}$',␣
 ↪fontsize=16, fontweight='bold', y=title_offset)

# --- Subplot (e) ---
## STEP 1. read sjf-ablation/[fcfs.pth  sjf.pth  suika-fcfs.pth  suika-sjf.pth]
## STEP 2. plot a clustered bar chart, where suika-fcfs/fcfs together,␣
 ↪suika-sjf/sjf together. (in JCT)
ax3 = plt.subplot(gs[1])
pos = ax3.get_position() # get current position [x0, y0, width, height]
ax3.set_position([
    pos.x0 + 0.011,  # shift right by 0.03
    pos.y0,
    pos.width,
    pos.height
])

groups = [('FIFO', ['suika-fcfs', 'fcfs']), ('SJF', ['suika-sjf', 'sjf'])]
group_labels = [g[0] for g in groups]
n_groups = len(groups)
x = np.arange(n_groups)
width = 0.35

policy_styles = {
    "suika-fcfs": {"color": "#34495e", "label": "FIFO + Suika"},
    "fcfs":       {"color": "#e74c3c", "hatch": "-", "label": "FIFO"},
    "suika-sjf":  {"color": "#2980b9", "label": "SJF + Suika"},
    "sjf":        {"color": "#3498db", "hatch": "-", "label": "SJF"},
}

# Plot bars for Suika policies
suika_jcts = [base_policy_stats[g[1][0]].get('jct', 0) for g in groups]
bars1 = ax3.bar(x - width/2, suika_jcts, width,
        color=[policy_styles[g[1][0]]['color'] for g in groups],
        edgecolor='white')
```

```
# Plot bars for base policies
base_jcts = [base_policy_stats[g[1][1]].get('jct', 0) for g in groups]
bars2 = ax3.bar(x + width/2, base_jcts, width,
        color=[policy_styles[g[1][1]]['color'] for g in groups],
        hatch=[policy_styles[g[1][1]]['hatch'] for g in groups],
        edgecolor='white')

ax3.set_ylabel('Average JCT (s)', fontsize=14, fontweight='bold')
ax3.set_xticks(x)
ax3.set_xticklabels(group_labels, fontsize=14)

# Add "+ Suika" text annotation
for i, bar in enumerate(bars1):
    yval = bar.get_height()
    ax3.text(bar.get_x() + bar.get_width()/2.0, yval + 50, '+ Suika',␣
 ↪ha='center', va='bottom', fontsize=12, fontweight='regular', rotation=60)

ax3.tick_params(axis='y', labelsize=12)
ax3.yaxis.set_label_position("left")
# ax3.yaxis.tick_right()
ax3.grid(True, axis='y', linestyle='--', alpha=0.7)
ax3.set_title('(e) Base Queueing Policy', fontsize=16, fontweight='bold',␣
 ↪y=title_offset)

# --- Subplot (f) ---
ax4 = plt.subplot(gs[2])
plot_metric_on_ax(
    ax4,
    trace_study_stats,
    'jct',
    'Average JCT (s)',
    'Trace',
    trace_map,
    '(f) Trace',
    policy_map,
    trace_study_policies,
    show_y_label=True,  # Enable Y label for subplot (f)
    sub_caption_y=title_offset
)
ax4.yaxis.set_label_position("left")
# ax4.yaxis.tick_right()
ax4.legend(fontsize=12, loc='upper left')

# --- Final Figure Adjustments ---
# fig.set_constrained_layout(True)
fig.tight_layout(rect=[0, 0.03, 1, 0.95])
```

```python
# plt.subplots_adjust(left=0.05, right=0.98, top=0.85, bottom=0.2, wspace=0.3)


# Save the figure
fig_dir = "../fig"
os.makedirs(fig_dir, exist_ok=True)
filename = f"{fig_dir}/sensitivity_study_row2.pdf"
plt.savefig(filename, dpi=300, bbox_inches='tight')
print(f"Combined plot saved as '{filename}'")

plt.show()
```

```
Loading data for sensitivity study…
  - Loaded lbd=0.25: JCT=3069.94, Util=67.82%
  - Loaded lbd=0.50: JCT=2637.67, Util=84.65%
  - Loaded lbd=0.75: JCT=2235.76, Util=96.13%
  - Loaded lbd=1.00: JCT=2127.41, Util=96.17%
  - Loaded lbd=1.25: JCT=2208.12, Util=96.30%
  - Loaded lbd=1.50: JCT=2167.58, Util=97.12%
  - Loaded lbd=1.75: JCT=2196.81, Util=96.91%
  - Loaded lbd=2.00: JCT=2190.96, Util=97.24%

Loading data for base policy sensitivity study…
  - Loaded fcfs: JCT=3561.93
  - Loaded sjf: JCT=3396.73
  - Loaded suika-fcfs: JCT=2127.41
  - Loaded suika-sjf: JCT=1940.87

Loading data for trace study…
  - Loaded clusterA/suika: JCT=2127.41
  - Loaded clusterA/sia: JCT=2735.18
  - Loaded clusterA/rubick: JCT=2786.77
  - Loaded clusterA/fcfs: JCT=3561.93
  - Loaded philly2/suika: JCT=2619.05
  - Loaded philly2/sia: JCT=3131.64
  - Loaded philly2/rubick: JCT=3545.82
  - Loaded philly2/fcfs: JCT=4854.63
  - Loaded helios_venus/suika: JCT=3831.95
  - Loaded helios_venus/sia: JCT=4365.27
  - Loaded helios_venus/rubick: JCT=4333.20
  - Loaded helios_venus/fcfs: JCT=6708.89
  - Loaded helios_uranus/suika: JCT=3232.36
  - Loaded helios_uranus/sia: JCT=3535.16
  - Loaded helios_uranus/rubick: JCT=3510.53
  - Loaded helios_uranus/fcfs: JCT=5938.49

/tmp/ipykernel_200624/3934630883.py:207: UserWarning: This figure includes Axes
that are not compatible with tight_layout, so results might be incorrect.
```

```
  fig.tight_layout(rect=[0, 0.03, 1, 0.95])
Combined plot saved as '../fig/sensitivity_study_row2.pdf'
```

